Actividad 2

Objetivo

Comprender los conceptos de la capa de enlace de datos (control de acceso al medio, direccionamiento MAC, detección de errores y entramado) mediante la simulación de una red con múltiples Arduinos que compiten por el medio.

Instrucciones generales

1. La Actividad puede ser realizada en parejas.
2. Lea y comprenda cuidadosamente lo que se le solicite.
3. La actividad está conformada por **Partes**. En cada Parte, podrá comprender ciertos conceptos teóricos dados en la lección.
   a) **Primera parte:** En esta parte, se creará una red con 3 Arduinos y se implementará un control de acceso al medio por contención (CSMA/CD simulado) para evitar colisiones.
   b) **Segunda parte:** Asimismo, en esta parte se modificará el código y la trama para incluir una dirección MAC de destino y origen, de manera que solo el receptor correcto procese el mensaje.
   c) **Tercera parte:** Finalmente, en esta parte se añadirá un mecanismo simple de detección de errores (Bit de Paridad) al final de la trama para verificar la integridad del mensaje.

Requisitos

1. Utilización del simulador Tinkercad. Disponible en el enlace https://www.tinkercad.com/dashboard.
2. Haber completado la Actividad 1 o tener conocimientos de programación en Arduino y manejo de pines.

Inicio de la Actividad.

Primera Parte – Control de Acceso al Medio (CSMA/CD Simulado)

1. Modifique el circuito: Vuelva a la simulación de la Tercera Parte de la Actividad 1 (la que tiene 2 Arduinos). Ahora, añada un tercer Arduino UNO R3 y un LED adicional con su respectiva resistencia de 100Ω. Conecte el LED de cada Arduino a su pin 13 (o el que prefieran). El medio de comunicación seguirá siendo el cable que conecta los pines digitales.
2. El problema de las colisiones: En el sistema actual, si dos Arduinos intentan hablar al mismo tiempo, las señales se mezclarán. Para simular esto, programaremos un comportamiento básico de "escuchar antes de hablar".
3. Programación del "Escuchar" (Lógica CSMA):
   a. Para TODOS los Arduinos (Emisores/Receptores): Antes de que un Arduino comience a enviar un mensaje, deberá "escuchar" el medio. ¿Cómo? Leyendo el pin por el que recibe datos. Si detecta un voltaje HIGH (lo que significa que otro Arduino está transmitiendo), deberá esperar un tiempo aleatorio y volver a intentar.

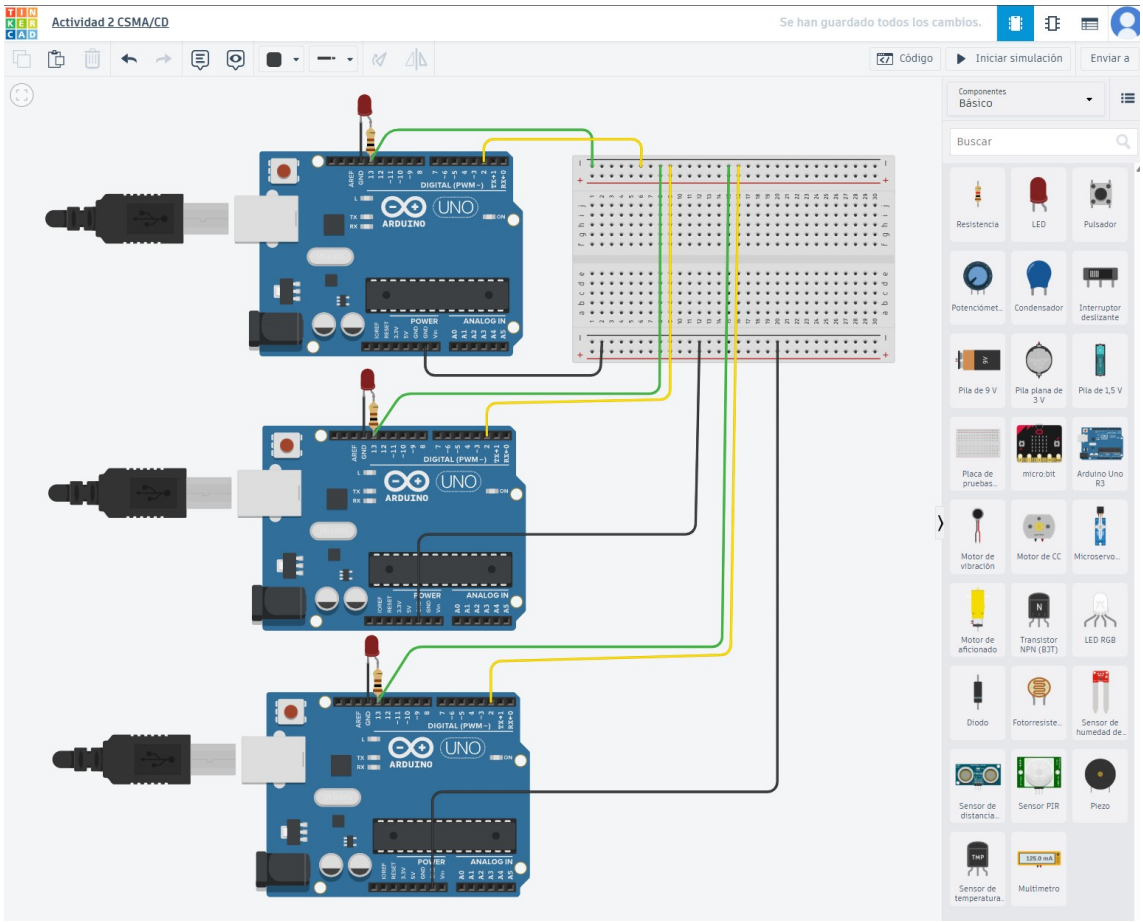

Figura 1. Autodesk Tinkercad. [1]

b. Implementación: Modifique el loop() del Arduino Emisor de la actividad anterior. *Puede utilizar IA, para acelerar el proceso de programación, pero recuerde que es su responsabilidad comprender y entender lo que le genera.*

4. Comprobación: Inicie la simulación. Abra los monitores serie de los dos Arduinos que actuarán como emisores. Envíe un mensaje desde ambos casi al mismo tiempo. Observe en el monitor serie cómo uno de ellos detecta el medio ocupado y espera antes de transmitir, simulando la prevención de colisiones.

Segunda Parte – Direccionamiento MAC

Ahora que tenemos 3 Arduinos en la red, cuando uno hable, los otros dos lo escucharán. Necesitamos una forma de que el mensaje llegue al destinatario correcto. Para ello, crearemos una "dirección MAC" simple.

1. Definición de la Trama: Diseñaremos una trama simple de capa de enlace con los siguientes campos:
   a. Inicio de Trama (Delimitador): Una secuencia especial para que los receptores se sincronicen. (Usaremos la señal de "Inicio de mensaje" de la actividad anterior: un pulso largo).
   b. Dirección MAC de Destino: Un número del 1 al 3 que identifica al Arduino que debe recibir el mensaje.
   c. Dirección MAC de Origen: Un número del 1 al 3 que identifica quién envía el mensaje.
   d. Datos: El mensaje codificado en 6 bits (como en la actividad anterior).
   e. Fin de Trama: Otra secuencia especial. (Usaremos la señal de "Finalizar mensaje").
2. Asignación de Direcciones MAC:
   a. Al primer Arduino, asígnele la dirección MAC 1.
   b. Al segundo Arduino, asígnele la dirección MAC 2.
   c. Al tercer Arduino, asígnele la dirección MAC 3. (Esto se define en el código con una variable, por ejemplo: uint8_t direccionMAC = 1;).
3. Modificación del Código del Emisor (Ej. Arduino MAC 1):
   a. Ahora, cuando se ingrese un mensaje por el monitor serie, el programa deberá preguntar también a quién va dirigido. (Ej: "Ingrese el mensaje: Hola" y luego "Ingrese MAC destino (1-3): 3").
   b. El mensaje se guardará en una variable msg, y la MAC destino en macDestino.

c. La función codificar() no solo codificará las letras, sino que primero transmitirá las direcciones. La secuencia de transmisión ahora será:
    1. Inicio de mensaje (pulso largo, igual que antes).
    2. Transmitir MAC Origen (1): Convertir el número 1 a su representación de 6 bits y enviar los pulsos.
    3. Transmitir MAC Destino (3): Convertir el número 3 a su representación de 6 bits y enviar los pulsos.
    4. Transmitir el mensaje ("Hola"): Letra por letra, como se hacía antes.
    5. Fin de mensaje (pulso extra largo, igual que antes).
d. Modificación del Código del Receptor:
    1. Todos los Arduinos receptores estarán siempre "escuchando".
    2. Al detectar el Inicio de mensaje, se prepararán para recibir.
    3. Leerán los dos primeros bloques de 6 bits: el primero es la MAC Origen y el segundo es la MAC Destino.
    4. Acción principal: El receptor compara la MAC Destino recibida con su propia direccionMAC.
    5. Si coincide, significa que el mensaje es para él. Entonces, procede a recibir el resto de los bits del mensaje, los decodifica y los imprime en su monitor serie.
    6. Si no coincide, ignora el resto de la transmisión y vuelve a escuchar. (Puede imprimir un mensaje como "Mensaje para otro nodo, ignorado...").
e. Comprobación:
    1. Inicie la simulación.
    2. Desde el Arduino MAC 1, envíe un mensaje a la MAC 2.
    3. Observe los monitores serie. El Arduino MAC 2 debería mostrar el mensaje decodificado. El Arduino MAC 3 (y el MAC 1, que también escucha) debería mostrar que el mensaje fue ignorado.
    4. Repita la prueba enviando mensajes a diferentes destinos.

Tercera Parte – Detección de Errores (Bit de Paridad)

En un entorno real, las señales pueden sufrir interferencias. Para simular esto, añadiremos un mecanismo simple de detección de errores: un bit de paridad.

1. Ampliación de la Trama: Modificaremos la trama para incluir un campo más justo antes del Fin de Trama:
   a. ...
   b. Datos: El mensaje codificado.
   c. Bit de Paridad: Un bit (un pulso) que sirve para la detección de errores.
   d. Fin de Trama.
2. Lógica del Bit de Paridad:
   a. Usaremos paridad par. Esto significa que el número total de bits '1' (pulsos HIGH) en los datos (incluyendo las direcciones y las letras del mensaje) debe ser par.
   b. En el Emisor: Después de ensamblar todos los bits a transmitir (MACs + Mensaje), el emisor cuenta cuántos bits '1' va a enviar. Si la cuenta es impar, el bit de paridad se envía como 1 (un pulso HIGH) para que el total sea par. Si la cuenta ya es par, el bit de paridad se envía como 0 (un período de LOW). Importante: En el sistema actual, un '1' es un pulso y un '0' es una pausa. El "bit de paridad" será una nueva señal que dura lo mismo que un punto/cero.
   c. En el Receptor: El receptor cuenta todos los bits '1' que recibe (MACs + Mensaje + Bit de Paridad). Si el total es par, asume que no hubo error (o que no lo detectó) y procede a decodificar y mostrar el mensaje. Si el total es impar, significa que hubo un error. Entonces, descarta el mensaje y muestra un mensaje de error: "¡Error de transmisión! Mensaje corrupto."
3. Implementación:
   a. Emisor: Modifique la función codificar() para que, después de enviar todos los bits del mensaje, calcule el bit de paridad y lo envíe (como un digitalWrite con un delay).
   b. Receptor: Modifique la función decodificar() o el proceso de recepción para que, además de reconstruir los bits en un String, cuente los '1's. Después de recibir el bit de paridad, verifique la paridad total antes de convertir los bits a letras.
4. Comprobación de Detección de Errores:
   a. Funcionamiento normal: Ejecute la simulación y envíe un mensaje. Debería funcionar con normalidad.
   b. Simulación de error: Para simular un error, podemos alterar el código del emisor temporalmente para que envíe un bit incorrecto. Por ejemplo, modifique el código de la letra 'A' para que el último bit sea el opuesto al que debería ser. Vuelva a ejecutar la simulación y observe cómo el receptor detecta la paridad incorrecta e imprime el mensaje de error, descartando el mensaje.

Actividad concluida satisfactoriamente.

Fin de la Actividad.

Referencias bibliográficas

[1] Captura de pantalla del simulador Tinkercad. Disponible en https://www.tinkercad.com/dashboard.